

Short Questions 9

Josh Manchester
Department of Computer Science
University of Colorado Colorado Springs
Colorado Springs, CO 80918
jmanches@uccs.edu

Abstract

This paper addresses three topics in deep reinforcement learning: the Decision Transformer architecture and its reframing of RL as sequence modeling, the advantage function and its benefits over raw action values, and two variants of Deep Q-Learning beyond Double DQN and Dueling DQN.

Question 1

How does the Decision Transformer work?

Most RL agents learn by fitting value functions or computing policy gradients. The Decision Transformer does neither. It takes a dataset of past trajectories and trains a Transformer to predict the next action—the same way GPT predicts the next word in a sentence (Chen *et al.* 2021). This recasts RL as a sequence modeling problem.

The input sequence is built from three types of tokens, interleaved at each timestep: a *return-to-go* $\hat{R}_t = \sum_{t'=t}^T r_{t'}$ (the sum of all future rewards from step t onward), the state s_t , and the action a_t . Each token type gets its own linear embedding layer, and a learned positional embedding is added per timestep—not per token—so one timestep maps to three tokens. These are fed into a GPT-style causal Transformer with masked self-attention, meaning each token can only attend to tokens at the same or earlier positions (Chen *et al.* 2021).

Training is straightforward supervised learning on a fixed offline dataset. The model predicts actions and is trained with cross-entropy loss for discrete actions or mean-squared error for continuous ones. The agent never interacts with the environment during training. This is offline RL.

At test time, we tell the model what score we want. We set the initial return-to-go to a high target return (e.g., the maximum possible score) and feed it alongside the starting state. The model outputs an action, the agent executes it and observes a reward, and the return-to-go is decremented by that reward. This repeats autoregressively—each step appends new tokens and generates the next action. So by conditioning on a high target return, the model produces expert-level behavior. The approach sidesteps bootstrapping, TD updates, and the “deadly triad” entirely (Chen *et al.* 2021).

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Credit assignment happens through self-attention over the full sequence instead.

Question 2

What is Advantage for an action in a state? What motivates the use of the advantage function?

The advantage function tells us how much better a particular action is compared to the average in a given state. We define it as:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (1)$$

$Q^\pi(s, a)$ is the expected return from taking action a in state s and then following policy π . $V^\pi(s)$ is the expected return from state s under π , averaged across all actions. So the advantage subtracts out the baseline value of the state and isolates how much an action contributes on its own. For the optimal action under a deterministic policy, $A = 0$; for every suboptimal action, $A < 0$. By construction, the expected advantage across actions is zero: $\mathbb{E}_{a \sim \pi(s)}[A^\pi(s, a)] = 0$ (Wang *et al.* 2016).

But why use this instead of raw Q -values? The core motivation is separating *where we are* from *what we do*. In many states, the choice of action barely matters—the state is either good or bad regardless of what we pick. A standard Q -network has to learn a large absolute value for every action in these states even though the relative differences are tiny. The advantage strips out that shared baseline $V(s)$ and focuses learning on what actually matters: which action is better than the others (Wang *et al.* 2016).

This has a practical payoff. When the network updates $V(s)$ from any single action’s experience, that update improves the Q estimate for *every* action in that state at once. In environments with many similar-valued actions, this means faster and more stable convergence. The Dueling DQN architecture exploits this directly by splitting the network into two streams—one for $V(s)$ and one for $A(s, a)$ —then combining them to produce Q -values (Wang *et al.* 2016). The advantage function also shows up in policy gradient methods, where subtracting $V(s)$ as a baseline reduces variance in the gradient estimates (Sutton & Barto, Ch. 13).

Question 3

Briefly discuss two variants of Deep Q-Learning, beyond Double DQN and Dueling DQN.

Prioritized Experience Replay. Standard DQN samples transitions uniformly at random from the replay buffer. Every transition gets the same chance of being replayed, whether the agent already knows it well or not. Prioritized experience replay changes this by sampling transitions in proportion to their TD error (Schaul *et al.* 2016). Transitions where the agent’s prediction was far off—meaning there is more to learn—get replayed more often. The priority is set as $p_i \propto |\delta_i| + \epsilon$, where δ_i is the TD error and ϵ is a small constant so nothing has zero probability. However, this biased sampling changes the distribution of updates, so importance-sampling weights are applied to correct for it and keep learning stable. The result is that the agent spends its time on surprising or informative experiences rather than wasting updates on transitions it already understands (Hessel *et al.* 2018).

Distributional DQN (C51). Standard DQN learns a single number for each state-action pair: the *expected* return $Q(s, a)$. But an average can hide a lot. Distributional RL learns the full *distribution* of returns instead (Bellemare *et al.* 2017). The return is modeled as a categorical distribution over a fixed set of N_{atoms} evenly spaced values (called “atoms”) between a minimum and maximum. For each state-action pair, the network outputs a probability for each atom—a complete picture of what returns are possible, not just their mean. Training minimizes the KL divergence between the predicted distribution and a target distribution built from the Bellman equation. This richer signal gives the network more to work with, and it shows: Distributional DQN outperforms standard DQN on the Atari benchmark. It is also one of the six components that Hessel *et al.* (2018) combined into the Rainbow agent, which achieved state-of-the-art performance across all 57 Atari games.

AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

References

- Bellemare, M. G.; Dabney, W.; and Munos, R. 2017. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 449–458.
- Chen, L.; Lu, K.; Rajeswaran, A.; Lee, K.; Grover, A.; Laskin, M.; Abbeel, P.; Srinivas, A.; and Mordatch, I. 2021. Decision Transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 34.
- Hessel, M.; Modayil, J.; van Hasselt, H.; Schaul, T.; Ostrovski, G.; Dabney, W.; Horgan, D.; Piot, B.; Azar, M.; and Silver, D. 2018. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 3215–3222.
- Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized experience replay. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. 2nd edition. MIT Press, Cambridge, MA.

Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 1995–2003.